

Helpdesk Use Cases

What every person can do in the Helpdesk app, start to finish, and what the app does when things go wrong.

Each use case opens with a short **summary** of what it is for. Then it gives what a tester needs: who does it, what must be true first, the steps, what else can happen, and what should be true at the end. It closes with the **API** routes behind it and **Checked by**, the end-to-end spec in [projects/helpdesk-e2e/src](#) that already walks through it, where one does.

The rules quoted here come from the shared contract ([projects/helpdesk-contract](#)) and the API, so the app, the API and this document agree. [How It Fits Together](#) shows how the parts connect.

Contents

1. [The people](#)
2. [What each role may do](#)
3. [Trying it](#)
4. [Rules that run through every use case](#)
5. Use cases
 - [Visitors and customers](#): UC-1 to UC-4
 - [Signing in](#): UC-5 and UC-6
 - [Agents](#): UC-7 to UC-11
 - [Supervisors](#): UC-12 to UC-18
 - [Admins](#): UC-19 to UC-21
 - [Across the app](#): UC-22 to UC-26
6. [Fields and limits](#)
7. [Messages people see](#)
8. [Security and privacy](#)
9. [Accessibility](#)
10. [Traceability](#)
11. [Glossary](#)

The people

Actor	Signs in?	What they do
Visitor	No	Reads the landing page and tries the demo
Customer	No	Reports an issue on the public site, with files if they like, and checks on it later
Agent	Yes	Takes unassigned tickets, moves them through the workflow, replies to customers and writes internal notes
Supervisor	Yes	Decides customer requests, assigns their team's tickets, follows each agent's work, and sees their team's reports

Actor	Signs in?	What they do
Admin	Yes	Creates and changes accounts, and sees reports for the whole help desk

Customers never sign in, so they are not a role. Each agent and supervisor belongs to one team; an admin belongs to none. One supervisor leads each team.

What each role may do

From `roles.ts` and the API's route guards (`@OnlyFor`), which enforce it:

Permission	Agent	Supervisor	Admin
Read a ticket and its conversation	Their own	Their team's	No
Reply to the customer, add an internal note	Their own	Their team's	No
Change a ticket's status	Their own	Their team's	No
Take an unassigned ticket	Yes, for themselves	No	No
Assign or reassign a ticket	No	To their team's agents	No
Decide customer requests	No	Yes, all of them	No
See reports	No	Their team and its agents	All
Create and change accounts	No	No	Yes
Download a customer's files	On their tickets	On requests and team tickets	No

Use case by role:

Use case	Visitor or customer	Agent	Supervisor	Admin
UC-1 Learn what the app does	✓			
UC-2 Report an issue	✓			
UC-3 Attach files to a report	✓			
UC-4 Check on a request	✓			
UC-5 Sign in		✓	✓	✓
UC-6 Sign out, or be signed out		✓	✓	✓
UC-7 Take an unassigned ticket		✓		
UC-8 Move a ticket through the workflow		✓	✓ (UC-16)	
UC-9 Read a ticket and its conversation		✓	✓	
UC-10 Reply, or add an internal note		✓	✓	
UC-11 Review recent work, and reopen		✓		
UC-12 Turn a request into a ticket			✓	
UC-13 Dismiss a request			✓	
UC-14 Assign and reassign tickets			✓	
UC-15 Follow a team member's work			✓	

Use case	Visitor or customer	Agent	Supervisor	Admin
UC-16 Change a team member's ticket status			✓	
UC-17 See a member's 3-month history			✓	
UC-18 See team reports			✓	
UC-19 Create an account				✓
UC-20 Change or deactivate an account				✓
UC-21 See reports for the whole help desk				✓
UC-22 See changes as they happen		✓	✓	✓
UC-23 Hear what happened		✓	✓	✓
UC-24 Files checked for viruses	✓			
UC-25 Try the demo	✓	✓	✓	✓
UC-26 Use it on a phone	✓	✓	✓	✓

Trying it

Command	Runs	Open
<code>yarn start:helpdesk</code>	The database in Docker, the API and the app from source, with instant reload	http://localhost:4200
<code>yarn start:helpdesk:docker</code>	All three as containers, built from the source	http://localhost:8088
<code>yarn start:helpdesk:docker:scan</code>	The same, plus ClamAV for the virus scan (UC-24)	http://localhost:8088
(nothing to run)	The in-browser demo (UC-25)	GitHub Pages

Each start seeds fresh data: four teams, each with a lead supervisor and 10 agents; 200 customers; 1,000 tickets; and a few waiting customer requests. Every seeded password is `helpdesk-dev-only`. Accounts used below:

User ID	Role	Team
<code>alex.morgan</code>	Admin	none
<code>chris.taylor</code>	Supervisor	leads Team Atlas
<code>sam.rivera</code>	Agent	Team Atlas
<code>omar.haddad</code>	Supervisor	leads Team Comet

Rules that run through every use case

Ticket statuses ([ticket.ts](#)): new, open, pending, resolved, closed. A move is allowed only if this table lists it. A resolved ticket can be reopened; a closed one is final.

From	May move to
New	Open, Closed
Open	Pending, Resolved, Closed

From	May move to
Pending	Open, Resolved, Closed
Resolved	Open, Closed
Closed	nothing

Open work is new, open or pending. **Finished** is resolved or closed. A ticket counts as finished when it leaves open work; closing a resolved ticket doesn't change that time, and reopening starts again.

Priorities and due times: low, normal, high, urgent. A ticket made from a customer request is due within its priority's SLA, counted from when it was made:

Priority	Due within
Urgent	4 hours
High	1 day
Normal	3 days
Low	7 days

Past its due time, a ticket shows as overdue. Lists show the most urgent first: the soonest due time first, so overdue tickets lead.

Queues, not teams: a ticket belongs to a queue. **Unassigned**, the open work nobody holds, is one list every team picks from. A ticket's team is its assignee's.

Who may touch a ticket: the agent who holds it, or their team's supervisor. Anyone else is told it isn't there (404), so a ticket's existence isn't given away.

Sessions last one support shift: 8 hours.

Visitors and customers

UC-1 Learn what the app does

Summary: a visitor sees what the help desk is for before signing in or reporting anything.

- **Actor:** visitor
- **Main flow:** they open the landing page (/). It shows a preview of a working day, how a ticket moves through its statuses, the three roles and their limits, and links to **Report an issue** and **Sign in**.
- **Alternatives:** an address that matches no page goes to the landing page.
- **API:** none; a first visit asks `GET /api/auth/me`, which answers "nobody" without logging an error.

UC-2 Report an issue

Summary: a customer with no account tells the help desk about a problem and gets a reference to follow it with. No confirmation email is sent, so the reference on screen is how they find it again.

- **Actor:** customer
- **Preconditions:** none. The page is public at `/report`, linked as **Report an issue** in the toolbar.
- **Main flow:**

1. The customer opens **Report an issue**.
 2. They fill in **Your name** and **Email**.
 3. They choose **What is it about?**: Sign-in and account, Billing, Something is broken, Feature request, or Other.
 4. They choose how much it affects them: I'm blocked, It's slowing me down, or I have a question.
 5. They fill in **Subject** and **Describe the problem. Where it happened** (a page, an order or account number) is optional.
 6. They may attach files (UC-3).
 7. They tick the box agreeing to how the request is kept.
 8. They select **Send request**.
- **Expected result:** the page says "We've got it. Your reference is R-..." and "Keep it: with your email, it's how you check on your request." The request waits in every supervisor's **New requests** (UC-12), and supervisors with the page open hear the arrival tone (UC-23).
 - **Alternatives and exceptions:**
 - **A field is missing or wrong:** the form says which, and nothing is sent. The email must look like an address (something, @, a domain with a dot, no spaces). Box not ticked: "Agree to how your request is kept, to send it."
 - **Sent faster than a person could, or the hidden field filled in:** a person takes at least 3 seconds to fill the form in, and never sees the hidden field. Such a request is answered as if it worked, and nothing is kept, so a bot can't tell it was caught.
 - **Too many from one address:** at most 5 requests an hour from one address. Past that: "Too many requests from here. Please try again in about ...".
 - **The service is down:** "Sending isn't available right now. Please try again in a few minutes."
 - **Rules:** the impact suggests the ticket's priority (blocked: high; slowing me down: normal; a question: low), and the category suggests its queue. Both are only suggestions: the supervisor decides (UC-12). A request is never deleted.
 - **API:** `POST /api/requests` (public; JSON, or `multipart/form-data` with files).
 - **Checked by:** `customer-requests.spec.ts`, "a visitor reports a problem, a supervisor makes it a ticket, an agent takes it".

UC-3 Attach files to a report

Summary: a customer adds screenshots or documents to show the problem. Pictures are redrawn on arrival, which strips hidden data such as where a photo was taken.

- **Actor:** customer
- **Preconditions:** filling in **Report an issue** (UC-2).
- **Main flow:**
 1. Under **Attachments (optional)**, the customer drops files on the drop zone, or picks them.
 2. Each file is listed with its name and a **Remove** button.
 3. They send the request (UC-2, step 8).
- **Expected result:** the files are kept with the request. Supervisors, and later the agent holding the ticket, see pictures as thumbnails and can download every file (UC-9, UC-12).
- **Alternatives and exceptions:**
 - **More than 3 files:** "Attach at most 3 files."

- **A file over 5 MB, or of a type not allowed:** refused, naming the file. Allowed: PNG, JPEG, WebP, PDF and plain text. The server decides a file's type from its contents, never from its name or what the browser says.
- **A picture that can't be redrawn,** including one over 40 megapixels: "... couldn't be read as an image."
- **Too many files from one address:** at most 25 MB of files an hour, so a person sending a few is never near it.
- **A file that fails the virus check:** UC-24.
- **Rules:** a picture is stored as redrawn: upright, its hidden data (location, camera, owner) gone. A file name over 200 characters is shortened. Staff open files only as downloads, never inside the page.
- **API:** POST /api/requests (the files part).
- **Checked by:** customer-requests.spec.ts , "a visitor reports with a photo, which the supervisor and the agent see and download, redrawn".

UC-4 Check on a request

Summary: a customer finds out what happened to their request, with no account, using its reference and their email.

- **Actor:** customer
- **Preconditions:** they sent a request (UC-2).
- **Main flow:**
 1. The customer opens **Check my request** (/report/status).
 2. They enter the **Reference** and the **Email** they sent it with, and select **Check**.
- **Expected result,** one of:
 - still waiting: "R-... is waiting for our team."
 - made a ticket: "R-... is now ticket #..., which is ...", with the ticket's current status
 - dismissed: "R-... was closed: ...", with the reason: Spam, Already reported, or Not a support request
- **Alternatives and exceptions:**
 - **No such reference, or the wrong email:** both get "No request matches that reference and email.", so references can't be tried one by one to learn who sent what.
 - **Typed loosely:** R-1042 , r-1042 , R1042 and 1042 all work, with spaces around them.
 - **Too many checks:** at most 30 in 10 minutes from one address: "Too many checks from here. Please try again in about ...".
 - **The service is down:** "Checking isn't available right now. Please try again in a few minutes."
- **API:** GET /api/requests/status (public).
- **Checked by:** customer-requests.spec.ts ; both journeys end with the visitor checking.

Signing in

UC-5 Sign in

Summary: a staff member signs in and lands on their own role's page.

- **Actor:** agent, supervisor or admin
- **Preconditions:** an active account.
- **Main flow:**

1. On the landing page, they select **Sign in**.
 2. They enter their user ID and password, and sign in.
- **Expected result:** agents land on `/agent`, supervisors on `/supervisor`, admins on `/admin`; a success tone plays (UC-23). Each page is open only to its own role.
 - **Alternatives and exceptions:**
 - **Wrong user ID or password:** "User ID or password is incorrect.", the same answer for both, so user IDs can't be guessed.
 - **Five failures for one user ID in 15 minutes:** that user ID is refused for a while, and the form says when to try again.
 - **A deactivated account:** can't sign in (UC-20).
 - **The service is down:** "Signing in isn't available right now. Please try again."
 - **Rules:** the session travels in a cookie the page's scripts can't read, and lasts 8 hours.
 - **API:** `POST /api/auth/sign-in`, `GET /api/auth/me`.
 - **Checked by:** `sign-in.spec.ts`.

UC-6 Sign out, or be signed out

Summary: staff leave when they choose, and are told plainly if their session ends while they work.

- **Actor:** agent, supervisor or admin
- **Main flow:** they select **Sign out** in the toolbar, and return to the landing page, which asks them to sign in again.
- **Alternatives: the session ends mid-work**, because the 8 hours ran out or an admin deactivated the account: their next action asks them to sign in again and says why.
- **API:** `POST /api/auth/sign-out`.
- **Checked by:** `sign-in.spec.ts` ("signs out to the landing page, which then asks to sign in again"), `session-ended.spec.ts`.

Agents

An agent's page has three lists: **My tickets** (their open work, most urgent first), **Unassigned** (the work nobody holds) and **Done (24 h)**.

UC-7 Take an unassigned ticket

Summary: an agent picks up waiting work for themselves.

- **Actor:** agent
- **Preconditions:** signed in as an agent; a ticket in **Unassigned**.
- **Main flow:** in **Unassigned**, the agent selects **Take it** on a ticket.
- **Expected result:** "#... is yours", a success tone, and the ticket moves to **My tickets**. A new ticket becomes open as it's taken.
- **Alternatives and exceptions:**
 - **Someone took it first:** the agent is told, and the ticket stays with whoever has it.
 - **It was finished meanwhile:** "This ticket is finished, so it can't be assigned."
 - **The service is down:** "Taking tickets isn't available right now. Please try again."

- **Rules:** an agent may take only unassigned tickets, and only for themselves.
- **API:** GET /api/tickets/unassigned , PUT /api/tickets/:ticketId/assignee .
- **Checked by:** agent.spec.ts , customer-requests.spec.ts .

UC-8 Move a ticket through the workflow

Summary: an agent records where a ticket is up to: waiting on the customer, resolved, or closed.

- **Actor:** agent (their supervisor too: UC-16)
- **Preconditions:** the ticket is theirs and not closed.
- **Main flow:**
 1. In **My tickets**, they open the ticket's **Change status** menu.
 2. The menu lists only the moves the workflow allows; they pick one.
- **Expected result:** the ticket shows its new status, with a success tone. Resolved or closed, it leaves **My tickets** for **Done (24 h)** (UC-11).
- **Alternatives:** a move the workflow doesn't allow is never offered; one sent anyway is refused, for example "An open ticket can't become new.". If the service is down: "Changing the status isn't available right now. Please try again."
- **API:** PUT /api/tickets/:ticketId/status .
- **Checked by:** agent.spec.ts , "an agent takes a ticket, moves it to Pending and replies".

UC-9 Read a ticket and its conversation

Summary: everything about one ticket in one place, kept current while it's open.

- **Actor:** the agent holding the ticket, or their team's supervisor
- **Main flow:** they select a ticket's subject in a list.
- **Expected result:** a popup shows the customer and their email, the status, the priority, who has it, and when it's due; then **Conversation:** the customer's first message (with **Files from the customer**, pictures as thumbnails, each with **Download**), then every reply and internal note, oldest first.
- **Alternatives:**
 - **The ticket changes while open:** status, assignee and due time update in place (UC-22); a new message appears and is read out by screen readers.
 - **It's given to someone else:** "Assigned to" says **Someone else**; who has it now isn't theirs to see.
 - **One click or many:** only one popup opens per ticket, however quickly it's clicked.
- **API:** GET /api/tickets/:ticketId , GET /api/tickets/:ticketId/messages , GET /api/tickets/:ticketId/attachments , GET /api/attachments/:id .
- **Checked by:** customer-requests.spec.ts (the agent opens the photo), live-updates.spec.ts .

UC-10 Reply to the customer, or add an internal note

Summary: replies are written to the customer; notes are for staff only.

- **Actor:** the agent holding the ticket, or their team's supervisor
- **Preconditions:** the ticket's popup is open (UC-9), and the ticket isn't closed.
- **Main flow:** they write in the box and select **Reply** or **Internal note**.
- **Expected result:** the message joins the conversation with their name and the time; a note stands apart, marked "Internal note".

- **Alternatives and exceptions:**
 - **Empty text:** both buttons stay off. Text is trimmed.
 - **Over 5,000 characters:** refused.
 - **A closed ticket:** takes no messages. A resolved one still does.
- **Rules:** a reply is kept on the ticket; no email is sent. The median time to the first reply is a report figure (UC-18); notes don't count toward it.
- **API:** `POST /api/tickets/:ticketId/messages`.
- **Checked by:** `agent.spec.ts`, `live-updates.spec.ts` (a note reaching the other browser).

UC-11 Review recent work, and reopen

Summary: an agent sees what they finished in the last day, and reopens a ticket that turns out not to be done.

- **Actor:** agent
- **Main flow:**
 1. **Done (24 h)** lists the tickets they resolved or closed in the last 24 hours.
 2. On a resolved one, **Change status** offers **Open**.
- **Expected result:** the reopened ticket returns to **My tickets**. A closed ticket can't be reopened.
- **Alternatives:** if the list can't load: "Your finished tickets couldn't be loaded.", with a way to try again.
- **API:** `GET /api/tickets/mine/finished`, `PUT /api/tickets/:ticketId/status`.
- **Checked by:** `agent.spec.ts`.

Supervisors

A supervisor's page has **New requests** at the top, then **My team** (pick a **Team member** to see their tickets), then **Unassigned**, with **Reports** beside the title.

UC-12 Turn a customer request into a ticket

Summary: a supervisor decides a waiting request is real work and makes it a ticket, with the queue and priority it needs.

- **Actor:** supervisor
- **Preconditions:** a request waiting in **New requests** (UC-2). Every supervisor sees all of them, oldest first.
- **Main flow:**
 1. The supervisor reads the card: reference, how long ago it came, category, impact, subject, the customer's name and email, where it happened, the description, and **Attached files**, pictures as thumbnails, each with a download button.
 2. **Possible duplicates** lists the same customer's open tickets and earlier requests, matched by email.
 3. They select **Turn into ticket**. The popup starts on the queue the category suggests and the priority the impact suggests; they may change either.
 4. They select **Create ticket**.
- **Expected result:** "R-... is now ticket #..., in Unassigned." The ticket is new and unassigned, due within its priority's SLA, with the request's subject and description, for the customer with that email (created, with the name they gave, if new). The customer's check now shows the ticket and its status (UC-4). The card leaves every supervisor's **New requests**.

- **Alternatives and exceptions:**
 - **Two supervisors decide the same request at once:** the first wins; the second is told it's already decided.
 - **The service is down:** "Deciding isn't available right now. Please try again."
- **API:** GET /api/requests , GET /api/queues , POST /api/requests/:requestId/ticket .
- **Checked by:** customer-requests.spec.ts .

UC-13 Dismiss a customer request

Summary: a supervisor closes a request that isn't support work, with a reason the customer sees.

- **Actor:** supervisor
- **Main flow:**
 1. On the request's card, they select **Dismiss**.
 2. They pick why: **Spam, Already reported** (with the number of the ticket it repeats), or **Not a support request**.
 3. They confirm.
- **Expected result:** "R-... dismissed." and the card leaves **New requests**. The request is kept, with who dismissed it and when. The customer's check shows "was closed:" and the reason (UC-4). Report figures count dismissals by reason (UC-18).
- **API:** POST /api/requests/:requestId/dismiss .
- **Checked by:** customer-requests.spec.ts , "a supervisor dismisses a request, and the visitor is told why".

UC-14 Assign and reassign tickets

Summary: a supervisor gives work to an agent on their team, or moves it to another.

- **Actor:** supervisor
- **Preconditions:** they lead a team ("You don't lead a team yet." otherwise).
- **Main flow:**
 1. On an **Unassigned** ticket, or one of a team member's tickets, they select **Assign**.
 2. **Assign to...** lists the team's active agents, each with their open and overdue counts; they pick one.
- **Expected result:** the ticket is that agent's, with a success tone, and it reaches the agent's open page at once, with the arrival tone there (UC-22, UC-23). Assigning a new ticket also opens it.
- **Alternatives and exceptions:**
 - **Nobody else on the team can take it:** "No other agent on your team can take it."
 - **A finished ticket:** "This ticket is finished, so it can't be assigned."
 - **A deactivated agent** isn't offered.
 - **The service is down:** "Assigning isn't available right now. Please try again."
- **Rules:** a supervisor assigns only to active agents on their own team, and only tickets that are unassigned or already their team's. Supervisors don't take tickets themselves.
- **API:** GET /api/teams/mine , PUT /api/tickets/:ticketId/assignee .
- **Checked by:** supervisor.spec.ts , "a supervisor assigns a ticket, reassigns it and checks the history"; live-updates.spec.ts .

UC-15 Follow a team member's work

Summary: a supervisor sees how loaded and how far behind each agent is.

- **Actor:** supervisor
- **Main flow:** under **My team**, **Team member** lists each active agent with their open and overdue counts (for example, "Benny Lind (5 open · 3 overdue)"). Picking one lists that agent's open tickets, most urgent first.
- **Alternatives:** a deactivated agent stays on the team but isn't listed. If the list can't load: "Your team couldn't be loaded."
- **API:** GET /api/teams/mine , GET /api/teams/mine/members/:userId/tickets .
- **Checked by:** supervisor.spec.ts .

UC-16 Change a team member's ticket status

Summary: a supervisor moves a ticket through the workflow for an agent, for example to resolve it while the agent is away.

- **Actor:** supervisor
- **Main flow:** on a team member's ticket, **Change status** offers the moves the workflow allows (UC-8).
- **Expected result:** as UC-8; the agent's page and any open popup follow the change (UC-22).
- **API:** PUT /api/tickets/:ticketId/status .
- **Checked by:** live-updates.spec.ts , "Chris resolves it: the popup says Resolved".

UC-17 See a member's 3-month history

Summary: a supervisor looks back over an agent's last three months.

- **Actor:** supervisor
- **Main flow:** next to a team member, they select **View 3-month history**.
- **Expected result:** a popup counts the agent's tickets created or changed in the period: assigned, finished, still open, finished on time and finished late; then lists them, most recently changed first.
- **Alternatives:** if it can't load: "This history couldn't be loaded."
- **API:** GET /api/teams/mine/members/:userId/history .
- **Checked by:** supervisor.spec.ts .

UC-18 See team reports

Summary: how the team's work is going: what's open, what's overdue, how fast it's finished and answered, and what customers are sending.

- **Actor:** supervisor
- **Main flow:**
 1. They select **Reports** beside the page title. With a team member picked, it opens on that agent.
 2. **Report for** switches between their team and any of its active agents.
- **Expected result:** for the team, charts of open work by priority and by status, and each agent's open work by priority; tiles for overdue work, work finished in the last 30 days, **SLA met** (finished by the due time), median hours to finish, and median hours to the first reply to the customer. The **Customer requests** card counts requests received, made tickets, dismissed by reason, and still waiting, with the median hours to a decision.

- **Rules:** open work is as of now; everything else covers the last 30 days. SLA met rounds down, so 100% means every ticket was on time. Customer requests figures are the same for every scope, as every supervisor sees all requests.
- **API:** `GET /api/reports` (`?agent=` for one agent).
- **Checked by:** unit specs in `projects/helpdesk/src/app/reports` and `projects/helpdesk-server/src/lib/reports` ; no end-to-end journey yet.

Admins

An admin's page, **Team accounts**, has one table per team (its lead marked "Supervisor · Lead", or "No lead"), then **Admins**.

UC-19 Create an account

Summary: an admin gives someone a sign-in, with their role and team.

- **Actor:** admin
- **Main flow:**
 1. They select **Create account**.
 2. They enter a user ID, a name, a role, a team (none for an admin: "Admins belong to no team.") and a starting password.
- **Expected result:** "Account ... created", and the account appears in its team's table, or in **Admins**. A new supervisor becomes their team's lead; the popup warns first, for example "Chris Taylor leads Team Atlas now. They'll stay on the team, but the new supervisor will lead it."
- **Alternatives and exceptions:**
 - **A user ID already taken:** "That user ID is taken."
 - **A malformed user ID:** it must be 3 to 32 characters (lowercase letters, digits, dots and hyphens), starting with a letter.
 - **A password under 12 or over 128 characters**, or a name over 100: refused.
 - **The service is down:** "Creating the account isn't available right now. Please try again."
- **Rules:** the server keeps only the password's hash.
- **API:** `GET /api/teams` , `POST /api/users` .
- **Checked by:** `admin.spec.ts` , `teams.spec.ts` .

UC-20 Change or deactivate an account

Summary: an admin moves someone to another team or role, or stops them signing in, without losing the work they held.

- **Actor:** admin
- **Main flow:**
 1. On the account's row, they select **Edit**.
 2. They change the role, the team, or whether the account is active, and save.
- **Expected result:** the account shows its new role, team and state ("Active: can sign in" or "Inactive: can't sign in").
- **Alternatives and exceptions:**

- **An agent who stops working a team's tickets** (deactivated, moved, or made a supervisor or admin): the popup warns "Any open tickets ... holds will go back to Unassigned.", and after saving the admin is told how many went back.
- **A deactivated account signed in right now:** their next action asks them to sign in again, and says why (UC-6).
- **A lead who stops leading:** the team has no lead until a supervisor joins it. An active account made a supervisor of a team, or moved to one as a supervisor, becomes its lead.
- **Their own account:** "You can't change your own account."
- **The last active admin:** "There must always be an active admin."
- **A team left with nobody:** stays listed, with no accounts and no lead, and can be joined again.
- **The service is down:** "Saving the account isn't available right now. Please try again."
- **API:** `GET /api/users` , `PUT /api/users/:userId` .
- **Checked by:** `admin.spec.ts` , `session-ended.spec.ts` , `teams.spec.ts` .

UC-21 See reports for the whole help desk

Summary: as UC-18, across every team.

- **Actor:** admin
- **Main flow:** **Reports**, then **Report for:** every team, any one team (with who leads it), or any active agent.
- **Expected result:** for every team, open work by team and priority, with an **Unassigned** row; otherwise as UC-18.
- **API:** `GET /api/reports` (`?team=` OR `?agent=`).

Across the app

UC-22 See changes as they happen

Summary: when work changes, everyone it concerns sees it without reloading.

- **Actors:** everyone signed in
- **Behavior:** taking, assigning, a status change, a reply or note, a new or decided request, and account changes each send a notice to the people they concern. An agent hears about their own tickets and Unassigned; a supervisor about their team's tickets, Unassigned, New requests and accounts; an admin about accounts. The open page then reloads what it shows through the normal API, so the stream carries no ticket data and can't get round a role check.
- **Alternatives:** if the API restarts, the pages reconnect and catch up. A stream ends with its session, and when the account changes.
- **API:** `GET /api/events` (server-sent events).
- **Checked by:** `live-updates.spec.ts` (two browsers at once), `api-restart.spec.ts` .

UC-23 Hear what happened

Summary: short tones confirm your own actions and flag new work arriving.

- **Behavior:**

Tone	When
Success (two rising notes)	Signing in, taking, assigning, a status change, deciding a request
Error (one low note)	Something refused, such as a wrong password
Arrival (one bright note)	Work or a message from someone else reaching your page, or a new customer request

Your own actions never set off the arrival tone. **Mute** in the toolbar silences all of them, remembered in that browser. The tones are made in code, with no sound files.

- **Checked by:** unit specs in `projects/helpdesk/src/app/sound ; live-updates.spec.ts` ("a ticket Chris assigns reaches Sam at once, with the arrival tone").

UC-24 Files checked for viruses (optional)

Summary: when the virus scan is switched on, every attached file is checked before it's kept.

- **Preconditions:** started with `yarn start:helpdesk:docker:scan`, which also runs ClamAV.
- **Expected result:** a clean file is kept as usual (UC-3).
- **Exceptions:**
 - **An infected file:** the request is refused, naming the file: "... didn't pass the virus check."; nothing is kept.
 - **The scan is on but can't check,** because ClamAV is unreachable or answers oddly: "Files can't be checked right now. Please try again in a few minutes." No file is kept unchecked.
- **Rules:** files are scanned as sent, before any picture is redrawn. Requests with no files are unaffected.
- **Checked by:** `virus-scan.spec.ts`, run with `E2E_SCAN=1`.

UC-25 Try the demo

Summary: anyone can try every role in their browser, with nothing to install.

- **Main flow:** open the [live demo](#). The bar at the bottom says "Preparing the demo..." while it loads, then offers **Try it as** Agent, Supervisor or Admin.
- **Expected result:** every page and every role, as in UC-1 to UC-21, with the Helpdesk's real services and rules running in the page against an in-browser PostgreSQL (PGLite), with the same migrations and seed data. Its field checks and role rules refuse what the real API refuses. Pictures are redrawn by the browser's canvas, dropping hidden data, and downloads work. Sounds for your own actions play, and **Mute** works.
- **What it leaves out:**
 - **Live updates** (UC-22): one tab, nobody to tell. Another tab or browser has its own database and sees none of your changes, and the arrival tone (UC-23) never plays.
 - **Anything lasting:** the data and the sign-in live in the tab's memory. A reload or a new visit starts over, signed out, with fresh data. Signing out and in as someone else within one visit keeps your changes.
 - **Rate and spam limits:** one visitor, so no limit on requests, checks or failed sign-ins, and no spam checks. The field checks (UC-2, UC-3) still apply.
 - **The virus scan** (UC-24): files are never scanned.
 - **Sessions and the web server:** no 8-hour session cookie (you stay signed in until you sign out or reload), and no nginx headers, caching or gzip.

UC-26 Use it on a phone

Summary: every page works on a phone.

- **Behavior:** the signed-in toolbar fits, with **Sign out** in reach; each role's page keeps to the screen, and wide tables scroll on their own.
- **Checked by:** `phone.spec.ts`.

Fields and limits

Field	Rule
User ID	3 to 32 characters: lowercase letters, digits, dots, hyphens; starts with a letter
Password	12 to 128 characters when set; only its hash is kept
Account name	1 to 100 characters
Customer name	Up to 100 characters
Customer email	Looks like an address; up to 254 characters
Where it happened	Optional; up to 200 characters
Ticket subject	Up to 200 characters
Ticket description	Up to 10,000 characters
Reply or note	1 to 5,000 characters, trimmed
Attachments	Up to 3 per request, 5 MB each; PNG, JPEG, WebP, PDF, plain text; pictures up to 40 megapixels
File name	Shortened past 200 characters
Request reference	<code>R-</code> and a number, typed back loosely
Ticket number	<code>#</code> and a number; unique, never reused

Limit	Value
Session	8 hours
Failed sign-ins	5 per user ID in 15 minutes
Customer requests	5 per address an hour
Files sent	25 MB per address an hour
Status checks	30 per address in 10 minutes
Shortest time to fill the form	3 seconds
Done list	the last 24 hours
Reports window	the last 30 days
Member history	the last 3 months

Messages people see

Situation	Message
Request sent	We've got it. Your reference is R-....
Too many requests	Too many requests from here. Please try again in about
Too many checks	Too many checks from here. Please try again in about

Situation	Message
Unknown reference, or the wrong email	No request matches that reference and email.
Too many files	Attach at most 3 files.
A picture that can't be redrawn	"..." couldn't be read as an image.
An infected file	"..." didn't pass the virus check.
The virus scan can't check	Files can't be checked right now. Please try again in a few minutes.
Wrong user ID or password	User ID or password is incorrect.
A finished ticket assigned	This ticket is finished, so it can't be assigned.
Nobody on the team to assign	No other agent on your team can take it.
A supervisor without a team	You don't lead a team yet.
A user ID in use	That user ID is taken.
An admin's own account	You can't change your own account.
The last admin	There must always be an active admin.
A service down	... isn't available right now. Please try again.

Security and privacy

- **Sign-in:** one answer for any failure; 5 failures per user ID in 15 minutes; sign-in accepts JSON only, so another site's form can't sign someone in. The session is an httpOnly cookie, never in a response body. The API refuses to start in production with the published development secret.
- **Access:** every route names the roles it allows; a test fails if one is left without. A ticket outside your reach answers as if it doesn't exist.
- **Customers:** no account, and no email sent. A reference only works with the email it was sent with. Spam is caught with a hidden field, a minimum fill time and per-address limits, with no paid service.
- **Files:** typed by their contents; pictures redrawn, dropping hidden data; downloads only, never shown inside the page; optionally scanned for viruses, refusing rather than keeping a file unchecked.
- **Passwords:** hashed with scrypt at OWASP's recommended cost.
- **Web server:** security headers on the app and the API, no server version given away; hashed scripts cached for a year, the page itself never; scripts gzipped (`web-server.spec.ts`).

Accessibility

- Every control has a name a screen reader can read, such as "Change status of [#1290](#)" or "Benny Lind, view 3-month history".
- New messages in an open conversation are read out as they arrive.
- The mute switch is labelled for screen readers.
- Tables scroll on their own on small screens, so a page never scrolls sideways (UC-26).

Traceability

Use case	End-to-end spec	Main API routes
UC-1	none	GET /api/auth/me
UC-2	customer-requests.spec.ts	POST /api/requests
UC-3	customer-requests.spec.ts	POST /api/requests
UC-4	customer-requests.spec.ts	GET /api/requests/status
UC-5	sign-in.spec.ts	POST /api/auth/sign-in
UC-6	sign-in.spec.ts , session-ended.spec.ts	POST /api/auth/sign-out
UC-7	agent.spec.ts , customer-requests.spec.ts	PUT /api/tickets/:ticketId/assignee
UC-8	agent.spec.ts	PUT /api/tickets/:ticketId/status
UC-9	customer-requests.spec.ts , live-updates.spec.ts	GET /api/tickets/:ticketId and its messages
UC-10	agent.spec.ts , live-updates.spec.ts	POST /api/tickets/:ticketId/messages
UC-11	agent.spec.ts	GET /api/tickets/mine/finished
UC-12	customer-requests.spec.ts	POST /api/requests/:requestId/ticket
UC-13	customer-requests.spec.ts	POST /api/requests/:requestId/dismiss
UC-14	supervisor.spec.ts , live-updates.spec.ts	PUT /api/tickets/:ticketId/assignee
UC-15	supervisor.spec.ts	GET /api/teams/mine
UC-16	live-updates.spec.ts	PUT /api/tickets/:ticketId/status
UC-17	supervisor.spec.ts	GET /api/teams/mine/members/:userId/history
UC-18	unit specs only	GET /api/reports
UC-19	admin.spec.ts , teams.spec.ts	POST /api/users
UC-20	admin.spec.ts , session-ended.spec.ts , teams.spec.ts	PUT /api/users/:userId
UC-21	unit specs only	GET /api/reports
UC-22	live-updates.spec.ts , api-restart.spec.ts	GET /api/events
UC-23	live-updates.spec.ts (arrival tone)	none
UC-24	virus-scan.spec.ts (E2E_SCAN=1)	POST /api/requests
UC-25	none (the demo's own unit specs)	answered in the page
UC-26	phone.spec.ts	none

Glossary

Term	Meaning
Request	What a customer sends through Report an issue; referenced as R-...

Term	Meaning
Ticket	A piece of support work, numbered #...
Unassigned	Open tickets nobody holds; every team picks from it
Open work	Tickets that are new, open or pending
Finished	Tickets that are resolved or closed
SLA	The time a ticket's priority allows before it's overdue
Lead	The one supervisor who leads a team
Internal note	A message on a ticket for staff only
Reply	A message on a ticket written to the customer, kept on the ticket
Possible duplicates	The same customer's open tickets and earlier requests, shown on a request's card